

```
public class SOLID {  
    @Override  
    public void implementPrinciples() {  
        // Código de alta calidad  
    }  
}
```



Principios **SOLID** en Java con POO



Single Responsibility | **O**pen-Closed | **L**iskov Substitution | **I**nterface Segregation | **D**ependency Inversion

Mejorando el diseño y la calidad del código orientado a objetos



 Con ejemplos prácticos en Java



```
interface CleanCode {  
    void maintain();  
    void extend();  
}
```

Introducción a los principios **SOLID**

Los principios SOLID son un conjunto de cinco directrices de diseño para la Programación Orientada a Objetos (POO), introducidos por Robert C. Martin ("Uncle Bob"):

S - **Single Responsibility Principle**

Principio de Responsabilidad Única

O - **Open/Closed Principle**

Principio Abierto/Cerrado

L - **Liskov Substitution Principle**

Principio de Sustitución de Liskov

I - **Interface Segregation Principle**

Principio de Segregación de Interfaces

D - **Dependency Inversion Principle**

Principio de Inversión de Dependencias

Propósito y Beneficios

Su propósito es guiar a los desarrolladores en la creación de software que sea comprensible, flexible, mantenible y escalable.

-  Código más limpio, con bajo acoplamiento y alta cohesión
-  Facilita la colaboración entre equipos de desarrollo
-  Mayor reutilización de código
-  Simplifica la adición de nuevas funcionalidades
-  Mantiene la estabilidad del sistema existente

"Dominar estos conceptos es fundamental para cualquier programador que desee mejorar la calidad de su código y construir aplicaciones robustas."

Principio de Responsabilidad Única (SRP)

Definición

El Principio de Responsabilidad Única establece que una clase debe tener **una, y solo una, razón para cambiar**. Esto significa que una clase debe tener una única responsabilidad o propósito bien definido dentro del software.

Propósito

El objetivo principal de este principio es:

- ✂ Reducir el **acoplamiento** entre componentes del sistema
- 🏠 Aumentar la **cohesión** dentro de cada clase
- 🔧 Facilitar el **mantenimiento** del código
- 🔑 Mejorar la **testeabilidad** de los componentes



Correcto

Una clase con un solo propósito es más fácil de entender y mantener

VS



Incorrecto

Una clase con múltiples responsabilidades es frágil y difícil de evolucionar

"Cuando una clase tiene una sola responsabilidad, un cambio en los requisitos afectará a una sola parte del sistema, minimizando el impacto y reduciendo el riesgo de errores."

SRP: Ejemplo de violación

Esta clase **Factura** viola el Principio de Responsabilidad Única al mezclar tres responsabilidades diferentes:



Lógica de negocio

Cálculo del total de la factura



Presentación

Impresión de la factura



Persistencia

Guardado en archivo

Razones para cambiar:

1. Si cambia la lógica de cálculo del total
2. Si cambia el formato de impresión
3. Si cambia el método de persistencia

```
// Clase Libro (utilizada por Factura)
class Libro {
    String nombre;
    String nombreAutor;
    int anyo;
    int precio;
    String isbn;

    public Libro(String nombre, String nombreAutor, int anyo, int precio, String isbn) {
        this.nombre = nombre;
        this.nombreAutor = nombreAutor;
        this.anyo = anyo;
        this.precio = precio;
        this.isbn = isbn;
    }
}

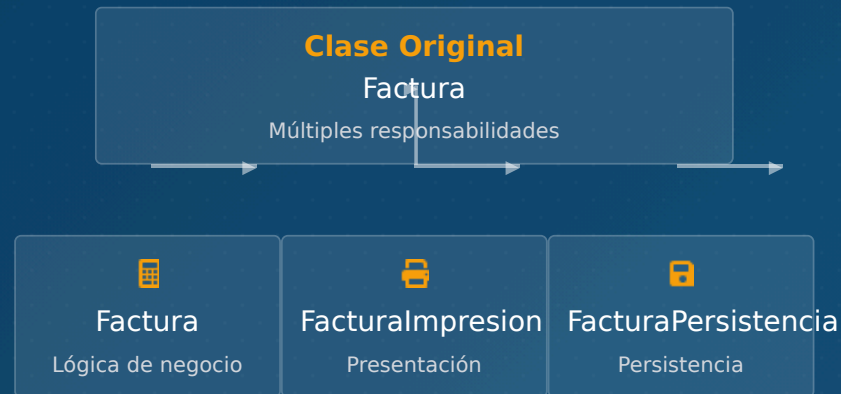
// Clase Factura que viola el SRP
public class Factura {
    private Libro libro;
    private int cantidad;
    private double tasaDescuento;
```

■ Lógica de negocio ■ Presentación ■ Persistencia

SRP: Implementación correcta

Para cumplir con el Principio de Responsabilidad Única, separamos las responsabilidades en clases distintas:

-  **Factura:** Solo lógica de negocio (cálculos)
-  **FacturaImpresion:** Solo presentación
-  **FacturaPersistencia:** Solo persistencia



```
// Clase Factura con una única responsabilidad (lógica de negocio)
public class Factura {
    private Libro libro;
    private int cantidad;
    private double tasaDescuento;
    private double tasaImpuesto;
    public double total;

    public Factura(Libro libro, int cantidad, double tasaDescuento, double tasaImpu
        this.libro = libro;
        // ...
    }
```

```
// Clase para la responsabilidad de impresión
class FacturaImpresion {
    private Factura factura;

    public FacturaImpresion(Factura factura) {
        this.factura = factura;
    }

    public void imprimir() {
        System.out.println(factura.getCantidad() + " unidades de "
            + factura.getLibro().nombre + " a " + factura.getLibro().precio
            + " cada una. Tasa de Descuento: " + factura.getTasaDescuento());
    }
}
```

```
// Clase para la responsabilidad de persistencia
class FacturaPersistencia {
    private Factura factura;

    public FacturaPersistencia(Factura factura) {
        this.factura = factura;
    }

    public void guardarArchivo(String nombreArchivo) {
        // Lógica para guardar la factura en un archivo
        System.out.println("Guardando factura en " + nombreArchivo);
    }
}
```

- ✓ **Beneficio clave:** Cada clase tiene una única responsabilidad. Si necesitamos cambiar el formato de impresión, solo modificaremos la clase FacturaImpresion, sin afectar a las demás.

Principio Abierto/Cerrado (OCP)

Definición

El Principio Abierto/Cerrado establece que las entidades de software (clases, módulos, funciones, etc.) deben estar:



Abiertas

para su **extensión**



Cerradas

para su **modificación**

Esto significa que deberíamos ser capaces de agregar nuevas funcionalidades sin alterar el código fuente existente, evitando introducir errores en código ya probado y en producción.

¿Cómo se logra?

Este principio se logra comúnmente mediante el uso de:

- 🧩 Abstracciones (interfaces y clases abstractas)
- 👤 Herencia y polimorfismo
- 🔌 Patrones de diseño como Strategy o Template Method



Modificar código ya probado puede introducir errores inesperados. Este principio permite que el sistema evolucione de manera más segura y mantenible.

OCP: Ejemplo de violación

⚠ Código que viola el Principio Abierto/Cerrado:

```
public class Coche {
    public String marca;

    public Coche(String marca) {
        this.marca = marca;
    }
}

public class Main {
    public static void imprimirPrecioCoche(Coche[] coches) {
        for (Coche coche : coches) {
            if (coche.marca.equals("Renault")) {
                System.out.println(18000);
            }
            if (coche.marca.equals("Audi")) {
                System.out.println(25000);
            }
        }
    }

    public static void main(String[] args) {
        Coche[] arrayCoches = {
            new Coche("Renault"),
            new Coche("Audi")
        };
        imprimirPrecioCoche(arrayCoches);
    }
}
```

OCP: Implementación correcta

```
// Clase base abstracta
public abstract class Coche {
    public abstract int getPrecio();
}

// Clases concretas que extienden la clase base
public class Renault extends Coche {
    @Override
    public int getPrecio() {
        return 18000;
    }
}

public class Audi extends Coche {
    @Override
    public int getPrecio() {
        return 25000;
    }
}

public class Mercedes extends Coche {
    @Override
    public int getPrecio() {
        return 27000;
    }
}
```

Principio Abierto/Cerrado

Las entidades de software deben estar:

 **Abiertas** para su extensión

 **Cerradas** para su modificación

Elementos clave de la solución:

 **Abstracción:** La clase base **Coche**

Principio de Sustitución de **Liskov** (LSP)

Definición y propósito

El Principio de Sustitución de Liskov, formulado por Barbara Liskov, establece que:

"Los objetos de una superclase deben poder ser reemplazados por objetos de sus subclases sin alterar el correcto funcionamiento del programa."

En otras palabras, si una clase **Hija** hereda de una clase **Padre**, cualquier código que utilice una instancia de **Padre** debería poder usar una instancia de **Hija** sin problemas ni comportamientos inesperados.

💡 El propósito principal es garantizar que una subclase extienda el comportamiento de su superclase sin reducirlo o modificarlo de manera incompatible.

Representación visual

```
class Padre { ... }
```



```
class Hija extends Padre { ... }
```

```
Padre p = new Hija();
```



La sustitución debe mantener el comportamiento esperado

Beneficios

- 🛡️ Evita errores sutiles y difíciles de detectar en la jerarquía de clases
- 👤 Asegura que la herencia modele una verdadera relación de "es un" en términos de comportamiento
- 🔑 Facilita el polimorfismo seguro y predecible
- 🧩 Mejora la reutilización y la coherencia del código

LSP: Ejemplo de violación

El ejemplo clásico del Rectángulo-Cuadrado demuestra cómo una jerarquía de herencia incorrecta puede violar el Principio de Sustitución de Liskov:

```
// Clase base
class Rectangulo {
    protected int ancho;
    protected int alto;

    public void setAncho(int ancho) {
        this.ancho = ancho;
    }

    public void setAlto(int alto) {
        this.alto = alto;
    }

    public int getArea() {
        return this.ancho * this.alto;
    }
}
```

```
// Subclase que viola el principio de Liskov
class Cuadrado extends Rectangulo {
    @Override
    public void setAncho(int ancho) {
        super.setAncho(ancho);
        super.setAlto(ancho); // Para mantener la propiedad del cuadrado
    }

    @Override
    public void setAlto(int alto) {
        super.setAlto(alto);
        super.setAncho(alto); // Para mantener la propiedad del cuadrado
    }
}
```

```
public class TestLSP {
    static void getAreaTest(Rectangulo r) {
        r.setAncho(5);
        r.setAlto(10);
        // Se espera que el área sea 5 * 10 = 50
        System.out.println("Área esperada: 50, "
            + "Área obtenida: " + r.getArea());
    }

    public static void main(String[] args) {
        Rectangulo rect = new Rectangulo();
        System.out.print("Probando con Rectangulo: ");
        getAreaTest(rect); // Área esperada: 50, Área obtenida: 50

        Rectangulo cuad = new Cuadrado();
        System.out.print("Probando con Cuadrado: ");
        getAreaTest(cuad); // Área esperada: 50, Área obtenida: 100
    }
}
```

¿Por qué viola LSP?

Cuando usamos un **Cuadrado** en lugar de un **Rectangulo**:

- Al llamar a setAlto(10)

LSP: Implementación correcta

Solución con interfaces

Para solucionar la violación del LSP, eliminamos la relación de herencia y usamos interfaces para modelar el comportamiento común.

- ✓ Cada clase implementa la interfaz de forma independiente, eliminando la dependencia jerárquica problemática.
- ✓ Las clases mantienen su comportamiento específico sin afectar a otras clases.
- ✓ Se cumple el principio: los objetos de una interfaz pueden ser sustituidos por implementaciones sin alterar el comportamiento esperado.

Enfoque alternativo

Crear jerarquías más específicas que reflejen relaciones "es un" correctas en términos de comportamiento, no solo de estructura.

Ejemplo: En un sistema de vehículos, una Bicicleta no debería heredar de VehiculoConMotor.

```
// Interfaz común para todas las figuras
interface Figura {
    int getArea();
}

// Clase Rectangulo implementa la interfaz
class RectanguloCorrecto implements Figura {
    private final int ancho;
    private final int alto;

    public RectanguloCorrecto(int ancho, int alto) {
        this.ancho = ancho;
        this.alto = alto;
    }

    @Override
    public int getArea() {
        return this.ancho * this.alto;
    }
}
```

Principio de Segregación de Interfaces (ISP)

Definición

"Los clientes no deberían verse forzados a depender de interfaces que no usan."

— *Interface Segregation Principle, Robert C. Martin*

Este principio establece que es preferible tener muchas interfaces **pequeñas y específicas** para cada cliente en lugar de una única interfaz grande y de propósito general.



Cuando una clase implementa una interfaz con métodos que no necesita, se ve obligada a proporcionar implementaciones vacías o a lanzar excepciones.

Representación conceptual

Interfaz Grande

</> método1()

</> método2()

</> método3()



Interfaz A

</> método1()

Interfaz B

</> método2()

Interfaz C

</> método3()

Beneficios

- ✓ Sistema más desacoplado y cohesivo
- ✓ Clases implementan solo los comportamientos relevantes
- ✓ Diseño más limpio, flexible y modular
- ✓ Menor probabilidad de errores por métodos no implementados

ISP : Ejemplo de violación

Problema: Interfaz "gorda"

Una interfaz que define demasiadas responsabilidades obliga a las clases a implementar métodos que no necesitan.

Violaciones del ISP:

 **Loro**: forzado a implementar `nadar()`

ISP: Implementación correcta

```
// Interfaz base con el comportamiento común
interface IAve {
    void comer();
}

// Interfaz para aves que pueden volar
interface IAveVoladora {
    void volar();
}

// Interfaz para aves que pueden nadar
interface IAveNadadora {
    void nadar();
}

// La clase Loro implementa solo las interfaces que necesita
class Loro implements IAve, IAveVoladora {
    @Override
    public void comer() {
        System.out.println("El loro está comiendo.");
    }
}
```

Segregación de Interfaces



Beneficios

- ✓ Cada clase implementa solo los métodos relevantes
- 🧩 Diseño más limpio y modular
- 🚫 Elimina implementaciones vacías o excepciones
- 🔑 Facilita la evolución de las interfaces
- 📦 Cumple con el Principio de Responsabilidad Única a nivel de interfaz

Principio de Inversión de Dependencias (DIP)

El Principio de Inversión de Dependencias (DIP), formulado por Robert C. Martin, establece que:

Regla 1:

Los módulos de alto nivel no deberían depender de los módulos de bajo nivel. Ambos deberían depender de abstracciones.

Regla 2:

Las abstracciones no deberían depender de los detalles. Los detalles (implementaciones concretas) deberían depender de las abstracciones.

Este principio busca **desacoplar los componentes** del software, invirtiendo el flujo de dependencia tradicional.

Beneficios:

- ✓ Código más flexible y mantenible
- ✓ Facilita las pruebas unitarias
- ✓ Permite sustituir implementaciones sin afectar a otros módulos

Dependencia Tradicional vs Invertida

❌ Dependencia Tradicional



✅ Dependencia Invertida (DIP)

Alto acoplamiento, difícil de mantener y probar



Bajo acoplamiento, fácil de mantener, testear y extender

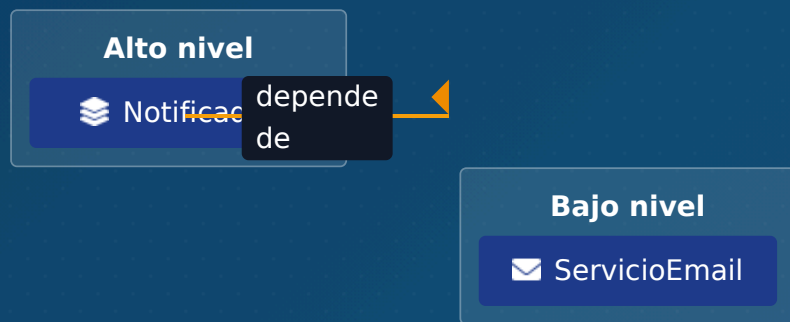
DIP: Ejemplo de violación

Problema

El Principio de Inversión de Dependencias establece que los módulos de alto nivel no deberían depender de los módulos de bajo nivel. Ambos deberían depender de abstracciones.

En este ejemplo, la clase **Notificador** (módulo de alto nivel) depende directamente de la clase concreta **ServicioEmail** (módulo de bajo nivel).

Visualización del problema



⚠ Acoplamiento fuerte

```
// Módulo de bajo nivel
public class ServicioEmail {
    public void enviarCorreo(String mensaje) {
        System.out.println("Enviando correo electrónico: " + mensaje);
        // Lógica para enviar un email
    }
}

// Módulo de alto nivel
public class Notificador {
    private ServicioEmail servicioEmail;

    public Notificador() {
        // El módulo de alto nivel crea una instancia directa del módulo de bajo nivel
        this.servicioEmail = new ServicioEmail();
    }

    public void enviarNotificacion(String mensaje) {
        servicioEmail.enviarCorreo(mensaje);
    }
}
```



Problemas de esta implementación:

16/18

- Acoplamiento fuerte entre el Notificador y el ServicioEmail
- Imposibilidad de cambiar el método de notificación sin modificar la clase Notificador
- Dificultad para realizar pruebas unitarias con mocks
- Viola el Principio Abierto/Cerrado al requerir modificaciones para extensión

DIP: Implementación correcta

Para cumplir con el Principio de Inversión de Dependencias, introducimos una **abstracción** (interfaz) de la que dependerán tanto el módulo de alto nivel como el de bajo nivel.

La dependencia se **inyecta** desde el exterior, permitiendo el intercambio de implementaciones sin modificar el código.

Estructura de dependencias:



💡 Tanto el módulo de alto nivel como los de bajo nivel dependen de la abstracción, invirtiendo el flujo de dependencia tradicional.

```
// Abstracción
```

```
public interface ServicioDeMensajeria {  
    void enviar(String mensaje);  
}
```

```
// Módulo de bajo nivel dependiendo de la abstracción
```

```
public class ServicioEmail implements ServicioDeMensajeria {  
    @Override  
    public void enviar(String mensaje) {  
        System.out.println("Enviando correo electrónico: " + mensaje);  
        // Lógica para enviar un email  
    }  
}
```

```
// Módulo de alto nivel dependiendo de la abstracción
```

```
public class Notificador {  
    private ServicioDeMensajeria servicio;  
    // La dependencia se inyecta desde el exterior  
    public Notificador(ServicioDeMensajeria servicio) {  
        this.servicio = servicio;  
    }  
    public void enviarNotificacion(String mensaje) {  
        servicio.enviar(mensaje);  
    }  
}
```

```
// Uso
```

```
public static void main(String[] args) {  
    // Usar el servicio de email  
    ServicioDeMensajeria servicioEmail = new ServicioEmail();  
    Notificador notificadorEmail = new Notificador(servicioEmail);  
    notificadorEmail.enviarNotificacion("¡Hola Mundo por Email!");  
    // Cambiar a servicio de SMS sin modificar la clase Notificador  
    ServicioDeMensajeria servicioSms = new ServicioSms();  
    Notificador notificadorSms = new Notificador(servicioSms);  
    notificadorSms.enviarNotificacion("¡Hola Mundo por SMS!");  
}
```

Conclusión y beneficios de SOLID

Los principios SOLID representan un conjunto de directrices fundamentales en la programación orientada a objetos que, al ser aplicados en Java, transforman significativamente la calidad del software.



Responsabilidad Única

Una clase, una responsabilidad



Abierto/Cerrado

Abierto a extensión, cerrado a modificación



Sustitución de Liskov

Las subclasses deben ser sustituibles por sus clases base



Segregación de Interfaces

Interfaces específicas son mejor que una general



Inversión de Dependencias

Depender de abstracciones, no de implementaciones

Beneficios clave



Código más limpio

Mayor legibilidad y reducción de complejidad a medida que los proyectos crecen.



Fácil de mantener

Bajo acoplamiento y alta cohesión facilitan las modificaciones y actualizaciones.



Código testeable

Componentes modulares e independientes facilitan las pruebas unitarias.



Flexibilidad

Adaptabilidad a nuevos requisitos sin comprometer el código existente.



Colaboración mejorada

Facilita el trabajo en equipo y la comprensión del código por nuevos desarrolladores.



Reutilización

Mayor capacidad para reutilizar componentes en diferentes partes del sistema.

"Aunque al principio puede parecer que seguir estas reglas añade un esfuerzo extra, la inversión se amortiza a largo plazo. Dominar los principios SOLID es una habilidad indispensable para cualquier desarrollador de Java que aspire a construir software de alta calidad, preparado para evolucionar y perdurar en el tiempo."